

UNIVERSIDADE DE SANTO AMARO
CURSO DE GRADUAÇÃO TECNOLÓGICA EM ANÁLISE
E DESENVOLVIMENTO DE SISTEMAS

PROJETO INTEGRADOR: DESENVOLVIMENTO DE SISTEMAS

Sistema Automatizado de Lembretes de Consultas por E-mail e WhatsApp

LORENÇO DE SOUSA GOMES

Apresentação do Projeto Integrador

Santo Amaro / SP
2026

Sistema Automatizado de Lembretes de Consultas por E-mail e WhatsApp

Trabalho de Projeto Integrador do curso de Análise e Desenvolvimento de Sistemas.

Área de concentração: Desenvolvimento de Sistema

Orientador(a): Prof.(a) Angelo Luiz da Cruz Oliveira

Santo Amaro / SP
2026

SUMÁRIO

Sumário

SUMÁRIO.....	3
RESUMO.....	5
ABSTRACT	5
1. INTRODUÇÃO	6
2. JUSTIFICATIVA.....	6
3. OBJETIVOS.....	7
3.1 Objetivo Geral.....	7
3.2 Objetivos Específicos.....	7
4. REQUISITOS DO SISTEMA.....	7
4.1 Requisitos Funcionais.....	7
4.2 Requisitos Não Funcionais	8
5. CASOS DE USO	8
5.1 Descrição dos Casos de Uso	8
5.2 Fluxo Principal (UC03 - Enviar lembrete)	8
6. DIAGRAMA DE FLUXO DO SISTEMA.....	9
7. MODELAGEM DO BANCO DE DADOS.....	10
7.1 Descrição das Tabelas	10
7.2 Controle de Duplicidade no Banco	10
7.3 Script SQL de Criação das Tabelas (schema.sql).....	11
8. ESTRUTURA DE PASTAS DO PROJETO	12
9. BIBLIOTECAS UTILIZADAS	12
9.1 Arquivo requirements.txt.....	12
10.IMPLEMENTAÇÃO DO SISTEMA	13
10.1 Configurações Gerais.....	13
10.2 Banco de Dados.....	13
10.3 Importação da Planilha	13
10.4 Envio de Notificações.....	13
10.5 Processamento dos Lembretes	13
10.6 Agendamento Automático.....	13
11. MECANISMOS DE QUALIDADE.....	13

11.1 Controle de Envio Duplicado	13
11.2 Registro de Histórico de Envios.....	14
11.3 Tratamento de Erros e Logs	14
11.4 Agendamento Automático.....	14
12. INSTALAÇÃO E EXECUÇÃO.....	14
12.1 Pré-requisitos	14
12.2 Passo a Passo.....	14
12.3 Modo de Simulação	15
12.4 Formato da Planilha Excel	15
13. TESTES REALIZADOS	15
13.1 Cenários e Resultados	15
13.2 Saída Registrada nos Logs (trecho)	16
14. RESULTADOS OBTIDOS	16
15. CONCLUSÃO	16
16. SUGESTÕES DE MELHORIAS FUTURAS	17
17. REPOSITÓRIO DO PROJETO	17
18. REFERÊNCIAS	17

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema automatizado de lembretes de consultas, construído na linguagem Python, com o objetivo de reduzir o índice de faltas (no-show) em clínicas e consultórios por meio do envio antecipado de notificações por e-mail e WhatsApp. O sistema lê os agendamentos a partir de uma planilha Excel, armazena os dados em um banco de dados SQLite e dispara, de forma automática, três lembretes para cada consulta: sete dias antes, vinte e quatro horas antes e três horas antes do horário marcado. Foram implementados mecanismos de controle de envio duplicado, registro de histórico, tratamento de erros, geração de logs e agendamento automático de execução. Os testes realizados em ambiente controlado demonstraram que o sistema identifica corretamente as janelas de tempo de cada lembrete e evita reenvios, comprovando sua aplicabilidade prática e seu potencial de contribuição para a organização do atendimento.

Palavras-chave: Python, automação, lembretes, SQLite, e-mail, WhatsApp.

ABSTRACT

This work presents the development of an automated appointment reminder system, built in the Python language, aiming to reduce the no-show rate in clinics and medical offices through the early sending of notifications by e-mail and WhatsApp. The system reads appointments from an Excel spreadsheet, stores the data in a SQLite database and automatically triggers three reminders for each appointment: seven days before, twenty-four hours before and three hours before the scheduled time. Mechanisms for duplicate-send control, history logging, error handling, log generation and automatic execution scheduling were implemented. Tests carried out in a controlled environment demonstrated that the system correctly identifies the time windows of each reminder and prevents resending, proving its practical applicability and its potential contribution to the organization of care.

Keywords: Python, automation, reminders, SQLite, e-mail, WhatsApp.

1. INTRODUÇÃO

Atualmente, muitas clínicas e consultórios enfrentam problemas com pacientes que faltam às consultas agendadas. Essas faltas podem causar prejuízos financeiros e dificultar a organização da agenda de atendimento.

Pensando nisso, este projeto apresenta um sistema automatizado de lembretes de consultas desenvolvido em Python. O sistema tem como objetivo enviar notificações por e-mail e WhatsApp antes da data da consulta, ajudando a reduzir o número de faltas.

O funcionamento é simples: os dados das consultas são lidos a partir de uma planilha Excel, armazenados em um banco de dados SQLite e utilizados para o envio automático dos lembretes em três momentos diferentes: sete dias, vinte e quatro horas e três horas antes da consulta.

Além do envio das mensagens, o sistema também registra informações sobre os lembretes enviados, evitando duplicidades e permitindo o acompanhamento das operações realizadas.

Palavras-chave: automação, lembretes de consultas, Python, redução de faltas, integração de canais.

2. JUSTIFICATIVA

A ausência de pacientes em consultas previamente agendadas representa um dos problemas mais recorrentes na gestão de serviços de saúde. Cada falta significa um horário ocioso que poderia ter sido ocupado por outro paciente, gerando perda de receita para o profissional e atraso no atendimento de quem aguarda na fila de espera. Estudos da área de gestão em saúde apontam que lembretes enviados de forma antecipada reduzem expressivamente esse índice de faltas.

A escolha por automatizar o processo se justifica pela inviabilidade de realizar lembretes manuais em larga escala. Ligar ou enviar mensagens individualmente para dezenas ou centenas de pacientes consome tempo da equipe e está sujeito a falhas humanas, como esquecimentos e envios duplicados. Um sistema automatizado executa essa tarefa de maneira padronizada, confiável e sem custo de mão de obra recorrente.

A utilização conjunta de e-mail e WhatsApp amplia o alcance da comunicação, uma vez que diferentes pacientes possuem preferências e hábitos distintos quanto ao canal de contato. O WhatsApp, em especial, apresenta altíssima taxa de abertura no Brasil, o que o torna um canal estratégico para esse tipo de notificação.

Do ponto de vista acadêmico, o projeto proporciona a aplicação prática de conhecimentos essenciais à formação do analista e desenvolvedor de sistemas, tais

como: modelagem e manipulação de banco de dados, leitura de arquivos externos, integração com APIs, tratamento de erros, geração de logs e agendamento de tarefas. Assim, o trabalho une relevância prática e valor educacional, preparando os envolvidos para os desafios reais do mercado de tecnologia.

3. OBJETIVOS

3.1 Objetivo Geral

Desenvolver um sistema automatizado, na linguagem Python, capaz de enviar lembretes de consultas por e-mail e WhatsApp em momentos predefinidos, contribuindo para a redução do índice de faltas e para a organização da agenda de atendimentos.

3.2 Objetivos Específicos

- Importar automaticamente os dados das consultas a partir de uma planilha Excel.
- Armazenar e organizar as informações de pacientes, consultas e envios em um banco de dados SQLite.
- Enviar lembretes automáticos sete dias, vinte e quatro horas e três horas antes de cada consulta.
- Integrar o sistema a serviços de e-mail (SMTP) e de WhatsApp (API da Twilio).
- Implementar um mecanismo de controle que impeça o envio duplicado de lembretes.
- Registrar um histórico completo de todos os envios realizados, com seus respectivos status.
- Tratar erros de execução e gerar logs detalhados para acompanhamento e auditoria.
- Permitir o agendamento automático da execução do sistema, sem intervenção manual.

4. REQUISITOS DO SISTEMA

Os requisitos foram divididos em funcionais, que descrevem o que o sistema deve fazer, e não funcionais, que descrevem características de qualidade e restrições.

4.1 Requisitos Funcionais

Código	Descrição
RF01	O sistema deve importar consultas a partir de uma planilha Excel.
RF02	O sistema deve cadastrar pacientes e consultas no banco de dados.
RF03	O sistema deve enviar lembretes por e-mail.

RF04	O sistema deve enviar lembretes por WhatsApp.
RF05	O sistema deve disparar lembretes 7 dias, 24 horas e 3 horas antes da consulta.
RF06	O sistema deve impedir o envio duplicado de um mesmo lembrete.
RF07	O sistema deve registrar o histórico de todos os envios e seus status.
RF08	O sistema deve executar-se automaticamente em intervalos definidos.

4.2 Requisitos Não Funcionais

Código	Descrição
RNF01	O sistema deve ser desenvolvido em Python 3.10 ou superior.
RNF02	O banco de dados deve ser leve e sem servidor (SQLite).
RNF03	As credenciais sensíveis devem ficar isoladas em um arquivo de configuração.
RNF04	O sistema deve registrar logs de execução em arquivo e em tela.
RNF05	Uma falha em um único registro não deve interromper o processamento dos demais.
RNF06	O código deve ser modular, comentado e de fácil manutenção.
RNF07	O sistema deve possuir um modo de simulação para testes sem envio real.

5. CASOS DE USO

O sistema possui dois atores principais: o Administrador (secretária ou profissional responsável pela agenda), que opera o sistema, e o Sistema de Agendamento, que executa as tarefas automáticas. O paciente é um ator externo, receptor das notificações.

5.1 Descrição dos Casos de Uso

Caso de Uso	Descrição
UC01 - Importar consultas	O administrador atualiza a planilha Excel e o sistema importa as novas consultas para o banco.
UC02 - Verificar lembretes devidos	O sistema percorre as consultas futuras e identifica quais lembretes devem ser enviados no momento.
UC03 - Enviar lembrete	O sistema envia o lembrete por e-mail e WhatsApp ao paciente correspondente.
UC04 - Controlar duplicidade	Antes de enviar, o sistema verifica se o lembrete já foi enviado e, em caso positivo, não repete.
UC05 - Registrar histórico	O sistema grava o resultado de cada envio (sucesso ou falha) no histórico.
UC06 - Agendar execução	O sistema executa-se automaticamente em intervalos definidos, sem intervenção humana.

5.2 Fluxo Principal (UC03 - Enviar lembrete)

1. O sistema obtém a lista de consultas futuras no banco de dados.

2. Para cada consulta, calcula quanto tempo falta até o horário marcado.
3. Verifica se esse tempo corresponde a uma das janelas de lembrete (7d, 24h ou 3h).
4. Consulta o histórico para confirmar que o lembrete ainda não foi enviado.
5. Monta a mensagem e a envia por e-mail e por WhatsApp.
6. Registra o resultado (sucesso ou falha) no histórico de envios.

6. DIAGRAMA DE FLUXO DO SISTEMA

O diagrama a seguir representa o fluxo lógico de execução do sistema, desde a inicialização do banco de dados até o registro do resultado dos envios. O processo é cíclico: a cada execução agendada, o sistema repete todas as etapas, garantindo que nenhum lembrete deixe de ser enviado no momento adequado.

Fluxograma do Sistema de Lembretes

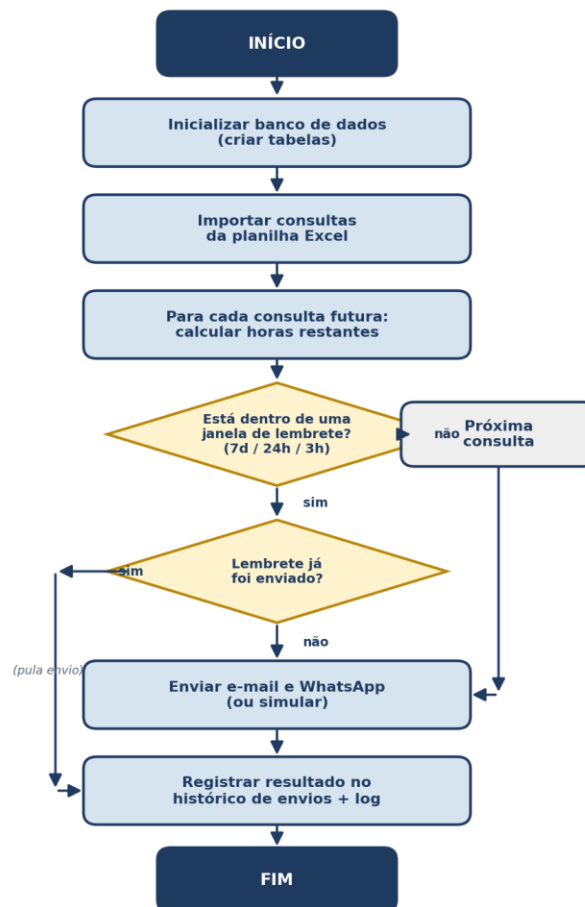


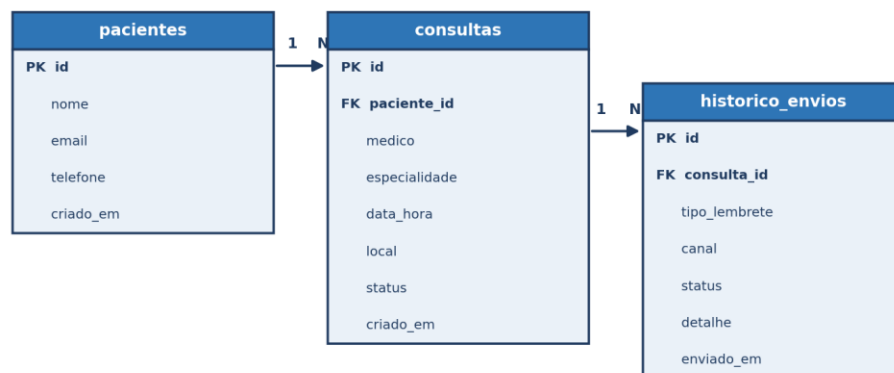
Figura 1 - Fluxograma do sistema de lembretes.

Destaca-se, no fluxo, a presença de duas estruturas de decisão fundamentais: a primeira verifica se a consulta está dentro de uma janela de lembrete; a segunda verifica se aquele lembrete específico já foi enviado. Essa segunda decisão é o coração do mecanismo de controle de duplicidade.

7. MODELAGEM DO BANCO DE DADOS

O banco de dados foi modelado com três tabelas relacionadas, utilizando o SGBD SQLite, que dispensa a instalação de um servidor e armazena todo o banco em um único arquivo. Essa característica torna o SQLite ideal para projetos de pequeno e médio porte e para fins didáticos.

Modelo do Banco de Dados (Diagrama Entidade-Relacionamento)



PK = chave primária FK = chave estrangeira

Figura 2 - Modelo Entidade-Relacionamento do banco de dados.

7.1 Descrição das Tabelas

- **pacientes:** armazena os dados de contato de cada paciente (nome, e-mail e telefone).
- **consultas:** registra cada agendamento, vinculado a um paciente por meio de uma chave estrangeira.
- **historico_envios:** guarda o registro de cada lembrete enviado, com tipo, canal e status.

O relacionamento entre as tabelas é do tipo um-para-muitos: um paciente pode ter várias consultas, e uma consulta pode gerar vários registros de envio (um para cada combinação de tipo de lembrete e canal).

7.2 Controle de Duplicidade no Banco

A tabela `historico_envios` possui uma restrição de unicidade (UNIQUE) sobre a combinação das colunas `consulta_id`, `tipo lembrete` e `canal`. Essa restrição é o mecanismo primário que impede, no próprio banco de dados, que o mesmo lembrete seja registrado mais de uma vez, garantindo a integridade dos dados mesmo diante de execuções repetidas.

7.3 Script SQL de Criação das Tabelas (schema.sql)

O script abaixo cria todas as tabelas e índices do sistema. Ele utiliza a cláusula `IF NOT EXISTS`, o que permite executá-lo várias vezes sem apagar dados já existentes.

```
-- =====
-- schema.sql
-- Script de criação das tabelas do banco de dados SQLite.
-- Sistema Automatizado de Lembretes de Consultas (E-mail e WhatsApp).
--
-- Para rodar manualmente:
--   sqlite3 lembretes.db < database/schema.sql
-- =====

-- Habilita verificação de chaves estrangeiras (desligada por padrão no SQLite).
PRAGMA foreign_keys = ON;

-----
-- Tabela: pacientes
-- Guarda os dados de contato de cada paciente.
-----
CREATE TABLE IF NOT EXISTS pacientes (
    id            INTEGER PRIMARY KEY AUTOINCREMENT, -- identificador único
    nome          TEXT    NOT NULL,                  -- nome completo
    email         TEXT,                                -- e-mail de contato
    telefone      TEXT,                                -- telefone no formato +55DDDDNUMERO
    criado_em     TEXT    DEFAULT CURRENT_TIMESTAMP  -- data de cadastro
);

-----
-- Tabela: consultas
-- Cada linha representa um agendamento de consulta.
-----
CREATE TABLE IF NOT EXISTS consultas (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    paciente_id   INTEGER NOT NULL,                  -- FK -> pacientes.id
    medico        TEXT,                                -- nome do profissional
    especialidade TEXT,                                -- ex.: Cardiologia
    data_hora     TEXT    NOT NULL,                  -- data/hora no padrão ISO (YYYY-MM-DD HH:MM)
    local         TEXT,                                -- endereço/sala da consulta
    status        TEXT    DEFAULT 'agendada',        -- agendada | realizada | cancelada
    criado_em     TEXT    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (paciente_id) REFERENCES pacientes(id)
);

-----
-- Tabela: historico_envios
-- Registra cada lembrete enviado. A restrição UNIQUE é o mecanismo
-- central de controle: impede que o mesmo lembrete (mesma consulta,
-- mesmo tipo e mesmo canal) seja enviado mais de uma vez.
-----
CREATE TABLE IF NOT EXISTS historico_envios (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    consulta_id   INTEGER NOT NULL,                  -- FK -> consultas.id
    tipo_lembrete TEXT    NOT NULL,                  -- '7d' | '24h' | '3h'
    canal         TEXT    NOT NULL,                  -- 'email' | 'whatsapp'
    status        TEXT    NOT NULL,                  -- 'sucesso' | 'falha'
    detalhe       TEXT,                                -- mensagem de erro ou id do provedor
    enviado_em    TEXT    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (consulta_id) REFERENCES consultas(id),
    UNIQUE (consulta_id, tipo_lembrete, canal)        -- evita envio duplicado
);
```

```
);

-- Índices para acelerar as consultas mais frequentes do sistema.
CREATE INDEX IF NOT EXISTS idx_consultas_data ON consultas (data_hora);
CREATE INDEX IF NOT EXISTS idx_envios_consulta ON historico_envios (consulta_id);
```

8. ESTRUTURA DE PASTAS DO PROJETO

O projeto foi organizado em módulos, separando responsabilidades em pastas distintas. Essa organização segue boas práticas de engenharia de software, facilitando a manutenção e a compreensão do código.

```
lembretes_consultas/
├── config.py           # configurações (SMTP, Twilio, regras)
├── main.py             # executa uma rodada completa
├── scheduler.py        # mantém o sistema rodando (a cada 1h)
├── testes.py           # testes de demonstração
├── requirements.txt    # dependências do projeto
├── consultas_exemplo.xlsx # planilha de exemplo
├── README.md           # instruções de uso
├── database/
│   ├── schema.sql     # criação das tabelas
│   └── db_manager.py  # acesso ao banco (DAO)
├── core/
│   ├── importador_excel.py # leitura da planilha
│   ├── agendador_lembretes.py # lógica dos lembretes + duplicidade
│   ├── enviador_email.py  # envio por e-mail (SMTP)
│   └── enviador_whatsapp.py # envio por WhatsApp (Twilio)
└── utils/
    └── logger.py       # configuração de logs
```

9. BIBLIOTECAS UTILIZADAS

O sistema utiliza um pequeno conjunto de bibliotecas, combinando recursos nativos do Python com bibliotecas externas amplamente consolidadas no mercado.

Biblioteca	Origem	Finalidade
sqlite3	Nativa	Comunicação com o banco de dados SQLite.
smtplib / email	Nativa	Montagem e envio de e-mails via protocolo SMTP.
logging	Nativa	Geração de logs em arquivo e em tela.
datetime	Nativa	Cálculo do tempo restante até cada consulta.
openpyxl	Externa	Leitura dos dados a partir da planilha Excel (.xlsx).
schedule	Externa	Agendamento automático da execução do sistema.
twilio	Externa	Envio de mensagens de WhatsApp pela API da Twilio.

9.1 Arquivo requirements.txt

As dependências externas ficam declaradas no arquivo requirements.txt, permitindo instalá-las todas de uma só vez com o comando `pip install -r requirements.txt`.

```
# Dependencias externas do projeto.
# Instale todas de uma vez com: pip install -r requirements.txt

openpyxl==3.1.5 # leitura/escrita de planilhas Excel (.xlsx)
```

```
schedule==1.2.2      # agendamento de tarefas em Python puro
twilio==9.3.7        # envio de WhatsApp via API da Twilio

# Observacao: o envio de e-mail usa as bibliotecas smtplib e email,
# que ja vem embutidas no Python (nao precisam ser instaladas).
```

10. IMPLEMENTAÇÃO DO SISTEMA

10.1 Configurações Gerais

O arquivo config.py centraliza as configurações do sistema, como caminhos de arquivos, credenciais de e-mail, configurações da API da Twilio e regras de envio dos lembretes.

10.2 Banco de Dados

O banco de dados utilizado foi o SQLite. Foram criadas três tabelas principais: pacientes, consultas e historico_envios. Essa estrutura permite armazenar os dados necessários para o funcionamento do sistema e registrar os lembretes enviados.

10.3 Importação da Planilha

A importação dos dados é realizada por meio da biblioteca OpenPyXL. O sistema lê os registros da planilha Excel e os cadastra automaticamente no banco de dados.

10.4 Envio de Notificações

Os lembretes podem ser enviados por e-mail e WhatsApp. O envio de e-mails utiliza o protocolo SMTP, enquanto o WhatsApp utiliza a API da Twilio. Durante os testes foi utilizado o modo de simulação para evitar envios reais.

10.5 Processamento dos Lembretes

O sistema verifica periodicamente as consultas cadastradas e identifica quais notificações devem ser enviadas. Os lembretes são disparados sete dias, vinte e quatro horas e três horas antes da consulta.

10.6 Agendamento Automático

A execução automática é realizada pela biblioteca Schedule, configurada para verificar periodicamente a existência de lembretes pendentes.

11. MECANISMOS DE QUALIDADE

11.1 Controle de Envio Duplicado

O controle de duplicidade é garantido em dois níveis. No nível do banco de dados, a restrição `UNIQUE` sobre `(consulta_id, tipo_lembrete, canal)` impede fisicamente o registro repetido de um mesmo lembrete. No nível da aplicação, antes de cada envio, a função `lembrete_ja_enviado()` consulta o histórico e, se o lembrete já tiver sido enviado com sucesso, o envio é simplesmente ignorado. Essa redundância torna o sistema robusto mesmo quando executado várias vezes seguidas.

11.2 Registro de Histórico de Envios

Cada tentativa de envio — bem-sucedida ou não — é registrada na tabela `historico_envios`, com o tipo de lembrete, o canal, o status e um campo de detalhe (que armazena, por exemplo, a mensagem de erro ou o identificador do provedor). Esse histórico permite auditoria, geração de relatórios e diagnóstico de problemas.

11.3 Tratamento de Erros e Logs

O sistema foi projetado para ser tolerante a falhas. Os envios de e-mail e WhatsApp são envolvidos por blocos `try/except` que capturam qualquer exceção e a registram no log, sem interromper o processamento das demais consultas. A importação da planilha trata erros linha a linha. Todos os eventos relevantes são gravados em arquivo de log e exibidos no terminal, com data, hora e nível de severidade.

11.4 Agendamento Automático

A execução periódica é responsabilidade do módulo `scheduler.py`, que utiliza a biblioteca `schedule` para repetir a rodada a cada hora. Essa frequência, combinada às tolerâncias definidas nas regras de lembrete, assegura que cada notificação seja disparada dentro da janela de tempo correta sem necessidade de intervenção manual.

12. INSTALAÇÃO E EXECUÇÃO

O passo a passo abaixo descreve como instalar e executar o sistema em qualquer computador com Python instalado.

12.1 Pré-requisitos

- Python 3.10 ou superior instalado.
- Acesso à internet (para envios reais por e-mail e WhatsApp).

12.2 Passo a Passo

7. Abrir o terminal na pasta raiz do projeto (`lembretes_consultas`).
8. Criar um ambiente virtual: `python -m venv venv`

9. Ativar o ambiente virtual (Windows: `venv\Scripts\activate` | Linux/Mac: `source venv/bin/activate`).
10. Instalar as dependências: `pip install -r requirements.txt`
11. Para executar uma única rodada: `python main.py`
12. Para manter o sistema rodando automaticamente: `python scheduler.py`

12.3 Modo de Simulação

No arquivo `config.py`, a constante `MODO_SIMULACAO` controla o comportamento de envio. Quando definida como `True`, o sistema apenas registra no log o que seria enviado, permitindo a apresentação do projeto sem credenciais reais. Para envios reais, basta defini-la como `False` e preencher as credenciais de e-mail (senha de app do Gmail) e da Twilio.

12.4 Formato da Planilha Excel

A planilha de entrada deve conter, na primeira linha, os seguintes cabeçalhos, e a coluna `data_hora` deve seguir o formato `AAAA-MM-DD HH:MM`:

paciente	email	telefone	medico	data_hora
Maria Oliveira	maria@email.com	+5511988887777	Dra. Ana	2026-06-15 14:30
Joao Pereira	joao@email.com	+5511977776666	Dr. Carlos	2026-06-09 09:00

Tabela 1 - Exemplo simplificado da planilha de entrada (colunas especialidade e local omitidas por espaço).

13. TESTES REALIZADOS

Para validar o sistema, foi criado o módulo `testes.py`, que gera consultas artificiais com horários calculados a partir do momento da execução (faltando 7 dias, 24 horas e 3 horas) e verifica se o sistema identifica e dispara os lembretes corretos. Os testes foram executados em modo de simulação.

13.1 Cenários e Resultados

ID	Cenário testado	Resultado	Observação
CT01	Importação de 4 consultas a partir do Excel	Aprovado	4 consultas cadastradas.
CT02	Disparo do lembrete de 7 dias	Aprovado	Identificado e enviado.
CT03	Disparo do lembrete de 24 horas	Aprovado	Identificado e enviado.
CT04	Disparo do lembrete de 3 horas	Aprovado	Identificado e enviado.
CT05	Reexecução do sistema (duplicidade)	Aprovado	Nenhum reenvio: lembretes pulados.

CT06	Consulta a 30 dias (fora da janela)	Aprovado	Nenhum lembrete disparado.
------	-------------------------------------	----------	----------------------------

13.2 Saída Registrada nos Logs (trecho)

O trecho abaixo, extraído do log durante a execução dos testes, evidencia tanto o envio correto dos três lembretes quanto o funcionamento do controle de duplicidade na segunda execução.

```
INFO - Consulta importada: Maria Oliveira em 2026-06-15 02:47.
INFO - Consulta 3: enviando lembrete '3h'.
INFO - [SIMULACAO] E-mail para ana.costa@email.com | faltam 3 horas
INFO - [SIMULACAO] WhatsApp para +5511966665555 | faltam 3 horas
INFO - Consulta 2: enviando lembrete '24h'.
INFO - Consulta 1: enviando lembrete '7d'.
INFO - Processamento concluído. 3 lembrete(s) tratado(s).
--- segunda execução ---
INFO - Lembrete 3h/email da consulta 3 já enviado. Pulando.
INFO - Lembrete 24h/whatsapp da consulta 2 já enviado. Pulando.
INFO - Lembrete 7d/email da consulta 1 já enviado. Pulando.
```

14. RESULTADOS OBTIDOS

Os resultados confirmaram que o sistema atende a todos os requisitos funcionais propostos. A importação da planilha cadastrou corretamente os pacientes e as consultas, sem gerar duplicidades em reexecuções. O mecanismo de cálculo de janelas identificou com precisão os três momentos de lembrete (7 dias, 24 horas e 3 horas), disparando as notificações nos dois canais previstos.

O controle de duplicidade mostrou-se eficaz: ao executar o sistema repetidamente, nenhum lembrete já enviado foi reenviado, comprovando a robustez da solução adotada nos níveis de banco de dados e de aplicação. O histórico de envios registrou fielmente cada operação, e os logs ofereceram visibilidade completa sobre o comportamento do sistema.

De maneira geral, o sistema demonstrou estabilidade, organização e aderência às boas práticas de desenvolvimento, cumprindo seu propósito de automatizar o envio de lembretes de consultas de forma confiável.

15. CONCLUSÃO

Com o desenvolvimento deste projeto foi possível aplicar diversos conhecimentos estudados durante o curso, como programação em Python, banco de dados, leitura de arquivos Excel, integração com APIs e automação de tarefas.

O sistema desenvolvido conseguiu importar consultas, armazenar os dados em banco de dados e enviar lembretes automáticos por e-mail e WhatsApp de acordo com as

regras definidas. Os testes realizados mostraram que o sistema funciona corretamente e evita o envio repetido de notificações.

Além de atender aos objetivos propostos, o projeto permitiu compreender melhor a importância da automação em processos do dia a dia, especialmente em ambientes que dependem de organização e comunicação com clientes ou pacientes.

16. SUGESTÕES DE MELHORIAS FUTURAS

- Desenvolver uma interface web (por exemplo, com Flask ou Django) para cadastro de consultas, eliminando a dependência da planilha Excel.
- Permitir respostas dos pacientes pelo WhatsApp para confirmar, cancelar ou remarcar a consulta automaticamente.
- Adicionar envio por SMS como terceiro canal de comunicação.
- Implementar um painel (dashboard) com indicadores de comparecimento, faltas e taxa de entrega das mensagens.
- Migrar o banco de dados para PostgreSQL ou MySQL em cenários com grande volume de dados.
- Hospedar o sistema na nuvem com execução agendada gerenciada, garantindo disponibilidade contínua.
- Personalizar os textos dos lembretes por especialidade ou por preferência do paciente.
- Adicionar testes automatizados com a biblioteca pytest e integração contínua.

17. REPOSITÓRIO DO PROJETO

O código-fonte completo, juntamente com os arquivos de configuração, documentação e exemplos utilizados durante o desenvolvimento, encontra-se disponível no repositório GitHub abaixo:

Disponível em: https://github.com/Loreco82/Projeto_Automacao

Acesso em: 15 jun. 2026.

18. REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR ISO/IEC 25010: engenharia de software — requisitos e avaliação da qualidade de produtos de software (SQuaRE) — modelos de qualidade. Rio de Janeiro: ABNT, 2011.

BANIN, Sérgio Luiz. Python 3: conceitos e aplicações — uma abordagem didática. São Paulo: Érica, 2018.

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. Introdução ao teste de software. Rio de Janeiro: Elsevier, 2007.

HEUSER, Carlos Alberto. Projeto de banco de dados. 6. ed. Porto Alegre: Bookman, 2009.

MENEZES, Nilo Ney Coutinho. Introdução à programação com Python: algoritmos e lógica de programação para iniciantes. 4. ed. São Paulo: Novatec, 2024.

PAULA FILHO, Wilson de Pádua. Engenharia de software: fundamentos, métodos e padrões. 3. ed. Rio de Janeiro: LTC, 2009.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.